Deployment and Github Repository:

Public Github Repository:

https://github.com/StevenBalagueraP/Lectio

Deployment Url:

https://lectio-red.vercel.app/

Testing — LectioFrontend

Documentación completa del conjunto de pruebas del proyecto. Incluye cómo correr las pruebas, descripción de cada suite, resultados reales de `npm test` y `npm run test:coverage`.

Herramientas utilizadas

Herramienta	Versión	Propósito
Jest	30.x	Test runner, mocks, cobertura
@testing-library/react	16.x	Renderizado de componentes en JSDOM
@testing-library/user-event	14.x	Simulación realista de eventos de usuario
@testing-library/jest-dom	7.x	Matchers de DOM (`toBeInTheDocument` , etc.)
babel-jest	30.x	Compilación de TypeScript / TSX para Jest
identity-obj-proxy	3.x	Mock de CSS Modules
jest-environment-jsdom	30.x	Entorno de navegador simulado

Cómo correr las pruebas

> Todos los comandos deben ejecutarse desde la carpeta **`LectioFrontend/`** .

```
# Correr todas las pruebas (una sola vez)
npm test

# Correr en modo watch (re-ejecuta al guardar archivos)
npm run test:watch

# Correr con reporte de cobertura
npm run test:coverage
```

Filtrar por archivo o suite

```
# Solo un archivo
npm test -- Pagination

# Solo un describe o it concreto
npm test -- --testNamePattern="shows an error"

# Solo pruebas de integración
npm test -- Page.test
```

Estructura de archivos de prueba

```
src/
├── test/                                  ← Infraestructura compartida
│   ├── setupTests.ts                      ← jest-dom + limpieza de localStorage
│   ├── mockFetch.ts                       ← Utilidades jsonResponse / emptyResponse / mockR
│   ├── envMock.ts                         ← Mock de import.meta.env (Vite → Jest)
│   ├── polyfills.ts                       ← Polyfills necesarios en JSDOM
│   ├── styleMock.ts                       ← Mock de archivos CSS planos
│   └── cookieEsMock.ts                    ← Mock de cookie-es (dependencia ESM de react-rou
│
├── shared/
│   ├── components/
│   │   ├── BookGrid/Bookgrid.test.tsx         UNITARIA
│   │   ├── Pagination/Pagination.test.tsx     UNITARIA (nueva)
│   │   └── RatingStars/RatingStars.test.tsx   UNITARIA (nueva)
│   └── hooks/
│       ├── useFetch.test.ts                   UNITARIA
│       └── useDebouncedValue.test.ts          INTEGRACIÓN (nueva)
│
└── features/
    ├── book-detail/components/LoanButton/Loanbutton.test.tsx   UNITARIA
    ├── book-reviews/
    │   ├── components/ReviewForm/Reviewform.test.tsx           UNITARIA
    │   └── pages/BookReviewsPage.test.tsx                      INTEGRACIÓN
    ├── categories/pages/CategoriesPage.test.tsx                INTEGRACIÓN (nueva)
    └── login/
        ├── validation.test.ts                                  UNITARIA (nueva)
        └── pages/LoginPage.test.tsx                            INTEGRACIÓN (nueva)
```

Pruebas Unitarias

Las pruebas unitarias prueban una única unidad en aislamiento (componente, hook o función) sin red real. Los únicos dobles son mocks de Jest para callbacks o `fetch`.

1. `BookGrid` — `Bookgrid.test.tsx`

Componente probado: `src/shared/components/BookGrid/BookGrid.tsx`

#	Caso	Tipo
1	Muestra el mensaje de error (y oculta libros) cuando hay un error	UI
2	Muestra 4 skeletons por defecto mientras carga sin datos	UI
3	Respeta `skeletonCount`	Props
4	Muestra el mensaje vacío por defecto cuando no hay libros	UI
5	Muestra un mensaje vacío personalizado vía `emptyMessage`	Props
6	Renderiza una tarjeta por libro	Renderizado
7	Convierte URLs de portada de `http` a `https`	Lógica
8	Llama `onOpen` con el id del libro al presionar "Ver detalle"	Interacción
9	Mantiene libros anteriores (atenuados) mientras carga una nueva página	Estado

2. `useFetch` — `useFetch.test.ts`

Hook probado: `src/shared/hooks/useFetch.ts`

#	Caso	Tipo
1	Pasa de `loading: true` a `data` al resolver	Ciclo de vida
2	Pasa un `AbortSignal` al fetcher	API
3	Expone el mensaje del error lanzado por el fetcher	Errores
4	Usa mensaje genérico cuando el rechazo no es un `Error`	Errores
5	No ejecuta el fetcher cuando `enabled` es false	Opciones
6	Deduplica peticiones en vuelo con la misma clave (TTL > 0)	Cache
7	Devuelve datos cacheados mientras el TTL no ha vencido	Cache
8	Vuelve a datos stale cuando el re-fetch falla	Cache

#	Caso	Tipo
9	Limpia los datos anteriores al cambiar la clave (`keepPrevious` = false)	Cache
10	Conserva datos anteriores al cambiar la clave con `keepPrevious` = true	Cache

3. `LoanButton — Loanbutton.test.tsx`

Componente probado: `src/features/book-detail/components/LoanButton/LoanButton.tsx`

#	Caso	Tipo
1–4	Muestra la etiqueta y el estado correcto para cada `LoanStatus`	Props / UI
5	Está deshabilitado y marcado como `aria-busy` durante la carga	Estado
6	Muestra "Status unavailable" cuando la petición falló	Error
7	Muestra "Status unavailable" cuando no hay estado todavía	Estado
8	Llama `onAction` con el status actual al hacer clic	Interacción
9	No llama `onAction` cuando el botón está deshabilitado	Interacción

4. `ReviewForm — Reviewform.test.tsx`

Componente probado: `src/features/book-reviews/components/ReviewForm/ReviewForm.tsx`

#	Caso	Tipo
1	Inicia sin rating seleccionado y Submit deshabilitado	Estado inicial
2	Selecciona un rating y habilita Submit	Interacción
3	Envía las estrellas y el comentario recortado	Submit
4	Permite un comentario vacío (es opcional)	Submit
5	No envía sin rating	Validación
6	Ignora submit disparado sin rating (bypass del botón deshabilitado)	Seguridad

#	Caso	Tipo
7	Ignora submit mientras ya hay una petición en curso	Estado
8	Actualiza el contador de caracteres y limita a 1000	UI
9	Deshabilita botones y muestra "Sending…" mientras envía	Estado
10	Muestra el mensaje de error recibido por props	Error
11	No muestra alerta cuando no hay error	UI
12	Llama `onCancel` al presionar Cancel	Interacción

5. `Pagination` — `Pagination.test.tsx` nueva

Componente probado: `src/shared/components/Pagination/Pagination.tsx`

#	Caso	Tipo
1	No renderiza nada cuando `totalPages` es 1	Condicional
2	No renderiza nada cuando `totalPages` es 0	Condicional
3	Deshabilita el botón "anterior" en la primera página	Estado
4	Deshabilita el botón "siguiente" en la última página	Estado
5	Marca la página actual con `aria-current="page"`	Accesibilidad
6	Llama `onChange` con `page - 1` al presionar "anterior"	Interacción
7	Llama `onChange` con `page + 1` al presionar "siguiente"	Interacción
8	Llama `onChange` con el número de página clicado	Interacción
9	Muestra todas las páginas sin elipsis cuando `totalPages` ≤ 7	Renderizado
10	Muestra elipsis al final cuando la página actual es baja en lista larga	Renderizado
11	Tiene un nav landmark accesible etiquetado "Paginación"	Accesibilidad

6. `RatingStars` — `RatingStars.test.tsx` nueva

Componente probado: `src/shared/components/RatingStars/RatingStars.tsx`

#	Caso	Tipo
1	Renderiza con `role="img"` y etiqueta accesible con el valor	Accesibilidad
2	Clampea valores menores a 0 a 0	Lógica
3	Clampea valores mayores a 5 a 5	Lógica
4	Acepta valores decimales	Lógica
5	Acepta el valor 0	Lógica
6	Acepta el valor 5	Lógica
7	Aplica la clase `lg` cuando `size="lg"`	Props
8	No aplica la clase `lg` cuando `size="sm"` (por defecto)	Props

7. `isValidEmail` / `validateLogin` — `validation.test.ts` nueva

Funciones probadas: `src/shared/utils/validation.ts` · `src/features/login/utils.ts`

#	Caso	Función
1	Retorna `true` para un email estándar	`isValidEmail`
2	Retorna `true` para email con subdominios	`isValidEmail`
3	Retorna `true` para email con plus addressing	`isValidEmail`
4	Retorna `false` si falta el `@`	`isValidEmail`
5	Retorna `false` si falta el dominio	`isValidEmail`
6	Retorna `false` si falta la parte local	`isValidEmail`
7	Retorna `false` para cadena vacía	`isValidEmail`
8	Ignora espacios al inicio/fin	`isValidEmail`

#	Caso	Función
9	Retorna objeto vacío para credenciales válidas	`validateLogin`
10	Reporta error de requerido cuando el email está vacío	`validateLogin`
11	Reporta error de formato cuando el email es inválido	`validateLogin`
12	Reporta error de requerido cuando la contraseña está vacía	`validateLogin`
13	Reporta ambos errores cuando email y contraseña están vacíos	`validateLogin`
14	Acepta email con espacios (trimming interno)	`validateLogin`

Pruebas de Integración

Las pruebas de integración montan la página real con router real y hooks reales. Solo se simula `fetch` (intercepción del `globalThis.fetch` con `mockFetch`).

8. `BookReviewsPage` — `BookReviewsPage.test.tsx`

Página probada: `src/features/book-reviews/pages/BookReviewsPage.tsx`

#	Caso
1	Carga el libro, el promedio y las reseñas
2	Muestra error con link a home cuando el libro no existe
3	Muestra mensaje cuando el libro no tiene reseñas
4	Muestra alerta cuando las reseñas no se pueden cargar
5	Redirige a `/login` cuando un usuario sin sesión presiona "Add Review"
6	Deja publicar una reseña a un usuario con sesión y refresca la lista
7	Envía `null` como comentario cuando el usuario lo deja vacío
8	Muestra el mensaje del servidor dentro del formulario si el backend rechaza la reseña

#	Caso
9	Pide volver a iniciar sesión cuando ésta venció (401)
10	Cierra el formulario sin enviar nada al presionar Cancel

9. `useDebouncedValue` — `useDebouncedValue.test.ts` nueva

Hook probado: `src/shared/hooks/useDebouncedValue.ts`

> Usa `jest.useFakeTimers()` para controlar el tiempo con precisión.

#	Caso
1	Devuelve el valor inicial inmediatamente sin esperar
2	No actualiza el valor antes de que transcurra el delay
3	Actualiza el valor después de que transcurra el delay
4	Reinicia el temporizador con cada nuevo valor (comportamiento debounce real)
5	Limpia el timeout al desmontar el componente
6	Respeta un delay personalizado

10. `CategoriesPage` — `CategoriesPage.test.tsx` nueva

Página probada: `src/features/categories/pages/CategoriesPage.tsx`

> Cada test usa un número de página distinto como query param para evitar colisiones con el cache en memoria.

#	Caso
1	Renderiza la lista de libros devuelta por la API al cargar
2	Muestra el contador de libros devuelto por el backend
3	Muestra alerta de error cuando la API responde con 500

#	Caso
4	Muestra el mensaje vacío cuando la API no devuelve libros
5	Abre el dropdown de categorías y lista todas las categorías
6	Navega a la página de detalle al presionar "Ver detalle"
7	Muestra el input de búsqueda con el placeholder correcto

11. `LoginPage — LoginPage.test.tsx` nueva

Página probada: `src/features/login/pages/LoginPage.tsx`

#	Caso
1	Renderiza el formulario con campos Email y Password
2	Muestra errores de validación en línea al enviar campos vacíos
3	Muestra error de "email inválido" para un email malformado
4	Muestra el mensaje de error del servidor cuando el backend responde con 401
5	Redirige a `/` después de un login exitoso
6	Muestra mensaje genérico de error cuando el backend responde con 500 sin detalle
7	Contiene un enlace a la página de registro
8	Muestra "Logging in…" y el botón deshabilitado mientras la petición está en curso

Resultados de `npm test`

```
PASS src/shared/hooks/useDebouncedValue.test.ts
PASS src/shared/components/RatingStars/RatingStars.test.tsx
PASS src/features/login/validation.test.ts
PASS src/shared/components/Pagination/Pagination.test.tsx
PASS src/features/book-detail/components/LoanButton/Loanbutton.test.tsx
PASS src/shared/hooks/useFetch.test.ts
PASS src/shared/components/BookGrid/Bookgrid.test.tsx
PASS src/features/book-reviews/components/ReviewForm/Reviewform.test.tsx
PASS src/features/categories/pages/CategoriesPage.test.tsx
PASS src/features/login/pages/LoginPage.test.tsx
PASS src/features/book-reviews/pages/BookReviewsPage.test.tsx

Test Suites: 11 passed, 11 total
Tests:       104 passed, 104 total
Snapshots:   0 total
Time:        ~4 s
```

Resultados de `npm run test:coverage`

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	77.45	70.02	78.43	81.09	
api	59.81	46.77	69.56	63.04	
http.ts	45.90	44.44	50.00	48.97	...18,125-126,144
token.ts	78.26	62.50	84.61	79.06	17,44-45,58-62,83
api/endpoints	70.83	25.00	63.63	77.27	
auth.ts	50.00	100.00	33.33	50.00	13-18
books.ts	68.75	25.00	66.66	78.57	22-24
categories.ts	100.00	100.00	100.00	100.00	
ratings.ts	100.00	100.00	100.00	100.00	
...nts/LoanButton	100.00	91.66	100.00	100.00	
LoanButton.tsx	100.00	91.66	100.00	100.00	28
...s/book-reviews	88.88	69.23	100.00	93.75	
hooks.ts	100.00	0.00	100.00	100.00	9
utils.ts	83.33	75.00	100.00	90.00	21
.../RatingSummary	100.00	100.00	100.00	100.00	
...nts/ReviewCard	100.00	100.00	100.00	100.00	
...nts/ReviewForm	100.00	100.00	100.00	100.00	
...nts/ReviewList	100.00	100.00	100.00	100.00	
...-reviews/pages	91.37	92.10	83.33	92.85	
BookReviewsPage	91.37	92.10	83.33	92.85	43-44,114,126
...res/categories	100.00	100.00	100.00	100.00	
...CategoryFilter	58.97	39.28	46.66	68.96	...32,35,47,87-99
...tegories/pages	50.00	41.93	33.33	54.05	...55-65,79-80,99
features/login	100.00	100.00	100.00	100.00	
utils.ts	100.00	100.00	100.00	100.00	
...es/login/pages	96.66	83.33	100.00	100.00	
LoginPage.tsx	96.66	83.33	100.00	100.00	31
...nents/AuthForm	100.00	100.00	100.00	100.00	
...nents/BookCard	100.00	100.00	100.00	100.00	
...nents/BookGrid	100.00	100.00	100.00	100.00	
...nents/BookHero	100.00	57.14	100.00	100.00	13-18
...ponents/Button	100.00	100.00	100.00	100.00	
...nts/Pagination	80.00	78.57	100.00	72.72	15-18
...ts/RatingStars	100.00	100.00	100.00	100.00	
...nents/Skeleton	100.00	0.00	100.00	100.00	7
...ents/TextField	100.00	85.71	100.00	100.00	14,28
shared/hooks	91.30	83.72	94.73	94.59	

```
  useBookDetail.ts |  100.00 |     0.00 |  100.00 |  100.00 | 6
  useLoanStatus.ts |  100.00 |    75.00 |  100.00 |  100.00 | 8
  useCategories.ts |  100.00 |   100.00 |  100.00 |  100.00 |
  useDebouncedVal. |  100.00 |     0.00 |  100.00 |  100.00 | 3
  useFetch.ts      |   95.83 |    94.28 |  100.00 |  100.00 | 38-41
  useGoBack.ts     |   40.00 |     0.00 |   50.00 |   50.00 | 8-9
  useSession.ts    |  100.00 |   100.00 |  100.00 |  100.00 |
 shared/utils      |   80.00 |    65.78 |   92.30 |   82.22 |
  apiError.ts      |  100.00 |   100.00 |  100.00 |  100.00 |
  book.ts          |   83.33 |    75.00 |  100.00 |  100.00 | 6
  cache.ts         |   82.35 |    72.22 |   87.50 |   81.25 | 19-21,54-56
  redirect.ts      |   40.00 |    36.36 |  100.00 |   50.00 | 4-5
  validation.ts    |  100.00 |   100.00 |  100.00 |  100.00 |
-------------------|---------|----------|---------|---------|-------------------

Test Suites: 11 passed, 11 total
Tests:       104 passed, 104 total
Snapshots:   0 total
Time:        ~7 s
```

Resumen de cobertura

Métrica	Resultado
Statements	77.45 %
Branches	70.02 %
Functions	78.43 %
Lines	81.09 %

Archivos con cobertura del 100 %

- `api/endpoints/categories.ts`
- `api/endpoints/ratings.ts`
- `features/book-reviews/components/ReviewForm/ReviewForm.tsx`
- `features/book-reviews/components/ReviewCard/ReviewCard.tsx`
- `features/book-reviews/components/ReviewList/ReviewList.tsx`
- `features/book-reviews/components/RatingSummary/RatingSummary.tsx`
- `features/login/utils.ts`
- `shared/components/AuthForm/AuthForm.tsx`

- shared/components/BookCard/BookCard.tsx
- shared/components/BookGrid/BookGrid.tsx
- shared/components/Button/Button.tsx
- shared/components/RatingStars/RatingStars.tsx
- shared/hooks/useCategories.ts
- shared/hooks/useSession.ts
- shared/utils/apiError.ts
- shared/utils/validation.ts

Convenciones del proyecto

Separación unitaria / integración

Tipo	Definición
Unitaria	Prueba una sola unidad (componente, hook, función) con mocks para todo lo externo. No hay red real.
Integración	Monta la página real con router real y hooks reales; solo se intercepta `globalThis.fetch`.

Estrategia de cache en pruebas de integración

El módulo `cache.ts` usa un `Map` en memoria que vive a nivel de módulo JS, por lo que persiste entre pruebas. Cada suite de integración usa claves de cache únicas por test:

- **`BookReviewsPage`** — Un `bookId` distinto por test (`nextBookId++`).
- **`CategoriesPage`** — Un número de página distinto en la querystring (`page=<n>`).

Mock de fetch

La función `mockFetch` del archivo `src/test/mockFetch.ts` reemplaza `globalThis.fetch` con un `jest.fn()`. Jest limpia los mocks automáticamente entre tests (`clearMocks: true` en `jest.config.cjs`). `localStorage` se limpia en `afterEach` en `setupTests.ts`.

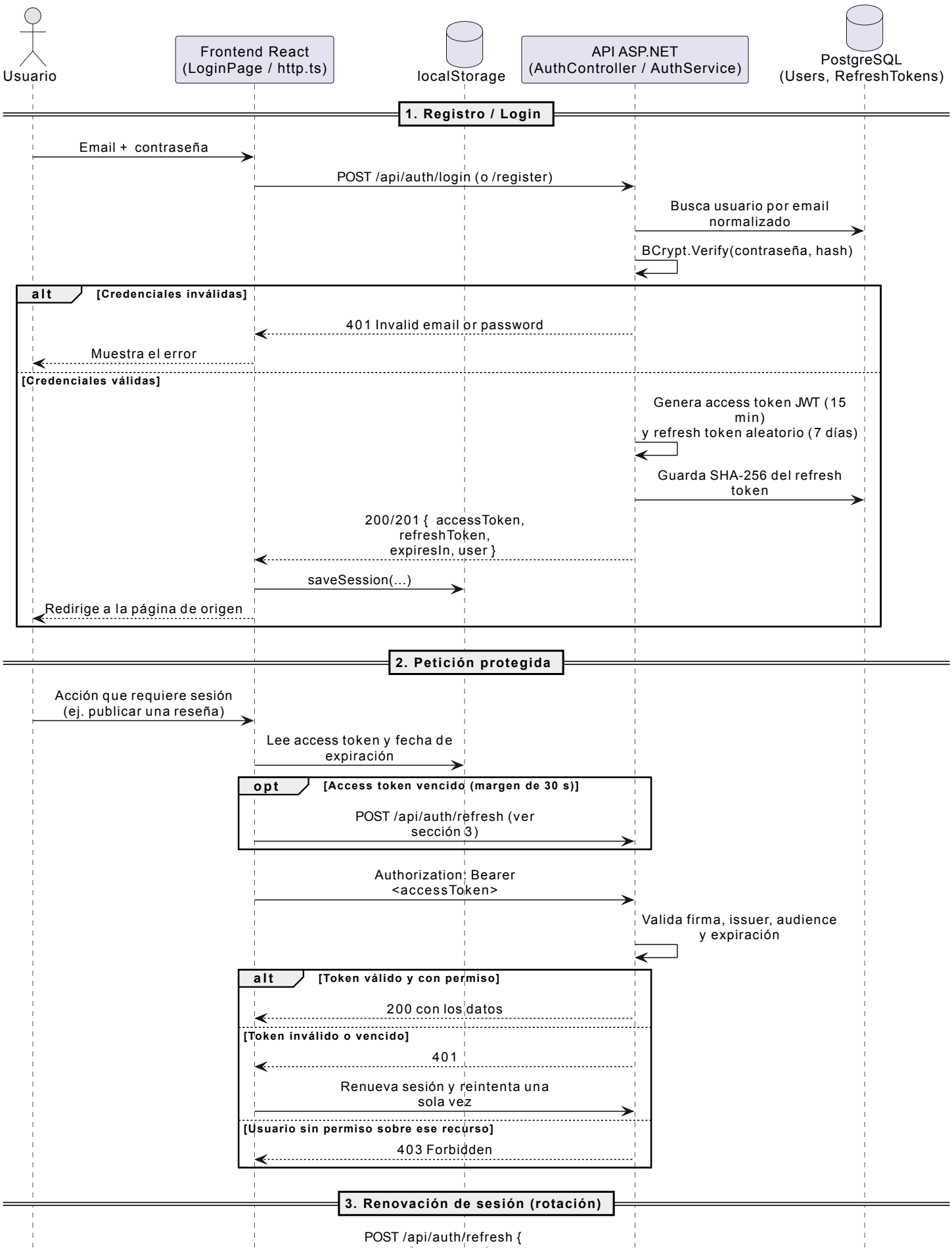
> Última actualización: octubre 2026

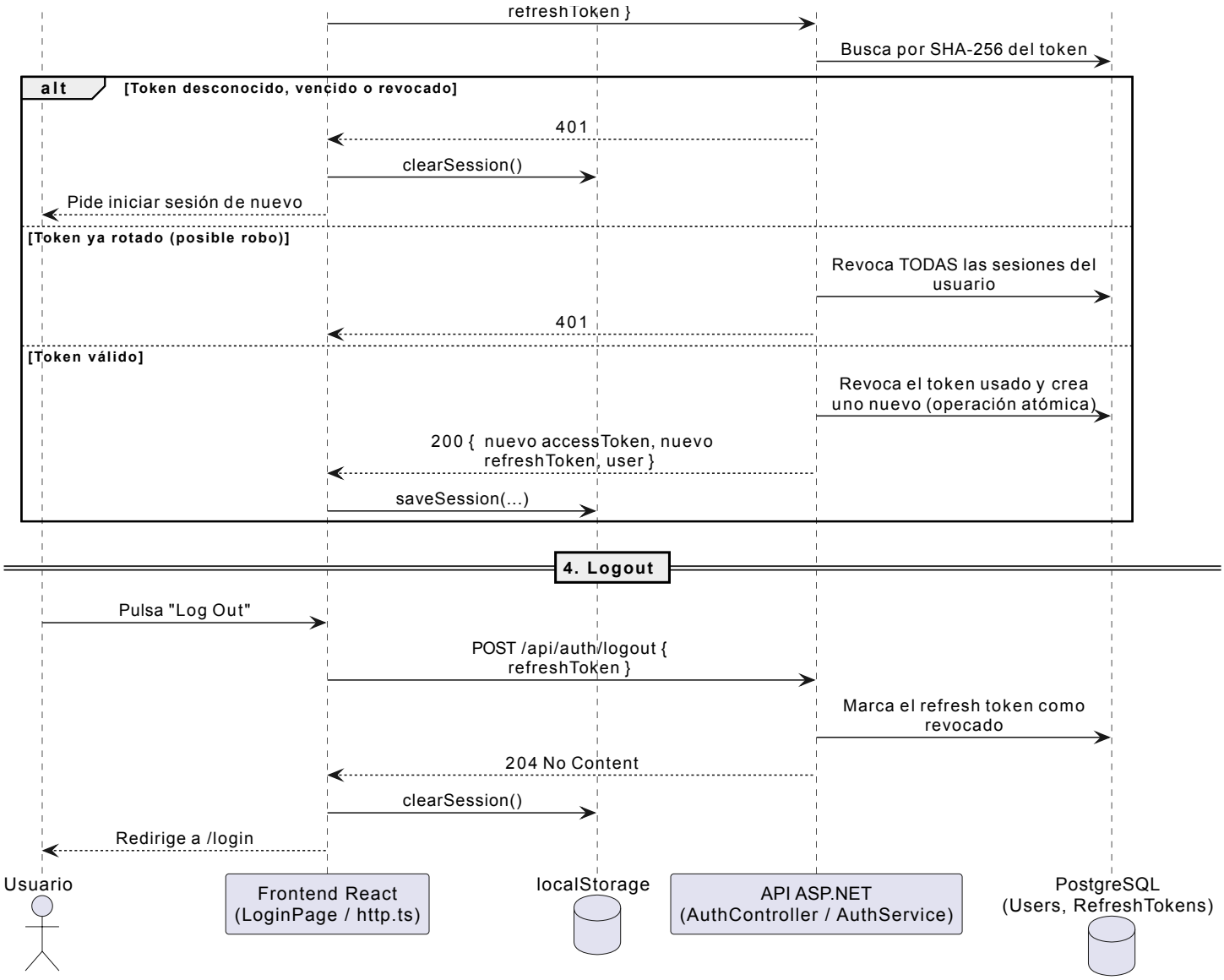
Autenticación en Lectio

Descripción breve

Lectio implementa un flujo de autenticación completo y seguro, basado en **JWT de corta duración** y **refresh tokens rotativos**. El frontend (React) se comunica con el backend (ASP.NET Core) y envía el token de autenticación en el encabezado `Authorization: Bearer <token>` en cada solicitud protegida. Si el token vence, el frontend lo renueva automáticamente sin que el usuario tenga que volver a iniciar sesión.

Diagrama (PlantUML)


Lectio - Flujo de autenticación (JWT + refresh token rotativo)
Usuario
Frontend React (LoginPage / http.ts)
localStorage
API ASP.NET (AuthController / AuthService)
PostgreSQL (Users, RefreshTokens)
1. Registro / Login
Email + contraseña
POST /api/auth/login (o /register)
Busca usuario por email normalizado
BCrypt.Verify(contraseña, hash)
alt [Credenciales inválidas]
401 Invalid email or password
Muestra el error
[Credenciales válidas]
Genera access token JWT (15 min) y refresh token aleatorio (7 días)
Guarda SHA-256 del refresh token
200/201 { accessToken, refreshToken, expiresIn, user }
saveSession(...)
Redirige a la página de origen
2. Petición protegida
Acción que requiere sesión (ej. publicar una reseña)
Lee access token y fecha de expiración
opt [Access token vencido (margen de 30 s)]
POST /api/auth/refresh (ver sección 3)
Authorization: Bearer <accessToken>
Valida firma, issuer, audience y expiración
alt [Token válido y con permiso]
200 con los datos
[Token inválido o vencido]
401
Renueva sesión y reintenta una sola vez
[Usuario sin permiso sobre ese recurso]
403 Forbidden
3. Renovación de sesión (rotación)
POST /api/auth/refresh {



Cómo funciona

El flujo tiene cuatro etapas, que corresponden a las secciones del diagrama.

Etapa	Qué pasa	Dónde está en el código
1. Registro / Login	El usuario envía su email y contraseña a `POST /api/auth/register` o `POST /api/auth/login`. El backend normaliza el email, verifica la contraseña con BCrypt y responde con un **access token** (JWT, 15 min), un **refresh token** (7 días) y los datos del usuario. En el registro se responde `201` y el usuario queda con sesión iniciada; si el email ya existe se responde `409`.	`AuthController`, `AuthService`, `TokenService` / `LoginPage`, `saveSession`
2. Petición protegida	El frontend lee el access token de `localStorage` y lo envía en `Authorization: Bearer`. Antes de enviar, si faltan menos de 30 s para que venza, lo renueva. El backend valida firma, emisor, audiencia y expiración. Si responde `401`, el frontend renueva la sesión y reintenta **una sola vez**.	`http.ts` (`getValidAccessToken`) / `AddJwtBearer` en `Program.cs`
3. Renovación	`POST /api/auth/refresh` recibe el refresh token, lo busca por su hash y, si es válido, lo **revoca y emite uno nuevo** (rotación). Si el token ya	`AuthService.RefreshAsync` / `doRefresh` en `http.ts`

Etapa	Qué pasa	Dónde está en el código
	había sido rotado, se asume que fue robado y se **revocan todas las sesiones** del usuario.	
4. Logout	`POST /api/auth/logout` revoca el refresh token en la base de datos (`204`). El frontend borra la sesión local siempre, incluso si no hay red.	`AuthService.LogoutAsync` / `useLogout`

Tokens

	Access token	Refresh token
Formato	JWT firmado con HMAC-SHA256	64 bytes aleatorios en Base64Url
Duración	15 minutos	7 días
Contenido	`sub`, `email`, `jti`, `fullName`, `role` y `phone` (si existe)	Sin información; es un valor opaco
Uso	En cada petición protegida (`Authorization: Bearer`)	Solo en `/api/auth/refresh` y `/api/auth/logout`
Almacenamiento en el servidor	No se guarda	Solo su hash SHA-256 (tabla `RefreshTokens`)
Almacenamiento en el cliente	`localStorage`	`localStorage`

Autorización

Una vez autenticado, el backend decide qué puede hacer cada usuario:

Recurso	Regla
`api/users/*` (préstamos y reseñas)	Requiere sesión (`[Authorize]`). Además, un usuario solo puede actuar sobre su propio `userId`; si intenta usar otro responde `403`, salvo que sea `admin`.
Controladores de seed	Solo rol `admin`.
`GET api/books/{id}/loan-status`	Es público, pero si llega un token válido el resultado se personaliza para ese usuario.
Catálogo de libros y reseñas	Públicos.

Medidas de seguridad

Medida	Detalle
Contraseñas con hash	BCrypt; nunca se guarda ni se devuelve la contraseña en texto plano.
Política de contraseña	Entre 8 y 72 caracteres, con al menos una letra y un número (72 es el límite de BCrypt).
Mensaje de error genérico	Login responde siempre `Invalid email or password`, sin revelar si el email existe. Se verifica contra un hash falso cuando el usuario no existe para que el tiempo de respuesta sea similar.
Rol no editable	El rol nunca viene del request: toda cuenta nueva se crea como `user`.
Refresh tokens hasheados	Si la base de datos se filtra, los hashes SHA-256 no sirven para iniciar sesión.
Rotación y detección de reutilización	Cada refresh token se usa una sola vez; reutilizarlo cierra todas las sesiones del usuario. La rotación es atómica, así que dos solicitudes simultáneas no pueden usar el mismo token.
Una sola renovación a la vez en el cliente	El frontend comparte una única renovación en curso y usa un lock entre pestañas (`navigator.locks`) para no gastar el refresh token dos veces.
Validación del JWT	Se verifican firma, `iss`, `aud` y expiración (tolerancia de 30 s). El servidor no arranca si `Jwt:Secret` tiene menos de 32 caracteres.
Límite de intentos	Login y registro: 5 solicitudes por minuto por IP. Global: 100 por minuto por IP.
CORS	Solo los orígenes configurados pueden llamar a la API.
Errores uniformes	Un middleware convierte las excepciones en respuestas `problem+json` (`400`, `401`, `409`, `500`).
Redirección segura	Tras iniciar sesión solo se aceptan rutas internas (`/...`), lo que evita redirecciones abiertas.

Consideraciones

- El access token ya emitido **no se invalida en el servidor** al cerrar sesión; deja de servir cuando vence (máximo 15 minutos). Por eso el frontend lo descarta al hacer logout.
- Los tokens se guardan en `localStorage`, que es accesible desde JavaScript. Es una decisión habitual en SPAs, pero hace que una vulnerabilidad XSS pueda exponerlos. Una alternativa más robusta sería enviar el refresh token en una cookie `HttpOnly`.